

Scenario2: (Week) Generate Test Scripts to verify reports section

Output:

Below are sample **UI Test Scripts** for verifying the **Reports Section** of an application. These are written from a Senior QA/Test Engineer perspective and can be automated using Selenium + Java/Cucumber/TestNG.

Reports Section Test Scripts

TC ID	Test Scenario	Steps	Expected Result
REP_001	Verify Reports page loads successfully	Navigate to Reports section	Reports page should load without errors
REP_002	Verify all report categories are displayed	Open Reports page	All configured report categories should be visible
REP_003	Verify default report selection	Open Reports page	Default report should be selected
REP_004	Verify report data is displayed	Select a report	Corresponding report data should appear
REP_005	Verify report generation with valid date range	Select From and To dates and click Generate	Report should be generated successfully
REP_006	Verify report generation with same start and end date	Select same date and generate report	Report should generate successfully
REP_007	Verify report generation with future date	Select future date range	Appropriate validation message should appear
REP_008	Verify From Date greater than To Date	Select invalid range	Error message should be displayed
REP_009	Verify report with no records available	Generate report for unavailable data	"No Data Available" message should be shown
REP_010	Verify report refresh functionality	Click Refresh button	Latest report data should load

TC ID	Test Scenario	Steps	Expected Result
REP_011	Verify report sorting ascending	Sort a column	Records should be sorted ascending
REP_012	Verify report sorting descending	Sort same column again	Records should be sorted descending
REP_013	Verify pagination functionality	Navigate between pages	Records should change accordingly
REP_014	Verify page size selection	Select 10, 25, 50 records/page	Selected number of records should be displayed
REP_015	Verify search functionality	Enter valid keyword	Matching records should be displayed
REP_016	Verify search with invalid keyword	Enter random text	No records message should appear
REP_017	Verify filters functionality	Apply filters	Only matching records should appear
REP_018	Verify multiple filters together	Apply multiple filters	Filtered data should be displayed correctly
REP_019	Verify filter reset option	Apply filters and click Reset	Default values should be restored
REP_020	Verify Export to Excel	Generate report and click Excel export	Excel file should download successfully
REP_021	Verify Export to PDF	Generate report and click PDF export	PDF file should download successfully
REP_022	Verify CSV export	Click CSV export	CSV file should be downloaded
REP_023	Verify downloaded file name format	Export report	File name should follow configured naming convention
REP_024	Verify exported report contents	Open downloaded file	Data should match UI records

TC ID	Test Scenario	Steps	Expected Result
REP_025	Verify special characters in exported data	Generate report containing special characters	Exported file should preserve characters
REP_026	Verify report print functionality	Click Print button	Print preview should open
REP_027	Verify column visibility settings	Hide/show columns	Grid should update accordingly
REP_028	Verify report data accuracy	Compare with database/API data	UI data should match backend data
REP_029	Verify report generation performance	Generate large report	Report should load within SLA
REP_030	Verify unauthorized user access	Login with restricted user	Access should be denied
REP_031	Verify role-based reports	Login with different roles	User should see permitted reports only
REP_032	Verify report after session timeout	Allow session to expire and access report	User should be redirected to login page
REP_033	Verify report loading indicator	Generate report	Loading spinner should appear and disappear after completion
REP_034	Verify report data persistence after page refresh	Refresh browser	Data should remain or reload as per requirement
REP_035	Verify browser back and forward navigation	Navigate using browser buttons	Reports page should behave correctly
REP_036	Verify report on different browsers	Execute on Chrome, Firefox, Edge	Behavior should remain consistent
REP_037	Verify report responsiveness	Open in different screen resolutions	UI should be properly aligned

TC ID	Test Scenario	Steps	Expected Result
REP_038	Verify report with maximum records	Generate report with large dataset	Application should not crash
REP_039	Verify concurrent report generation	Multiple users generate reports simultaneously	Reports should be generated successfully
REP_040	Verify error handling during server failure	Simulate API failure	User-friendly error message should be displayed

Download Specific Test Cases

Excel Download

Scenario: Verify Excel report download

Given user is on Reports page

When user selects valid date range

And clicks Generate Report

And clicks Export Excel

Then Excel file should be downloaded

And file extension should be ".xlsx"

And downloaded data should match UI records

PDF Download

Scenario: Verify PDF report download

Given user is on Reports page

When user generates report

And clicks Export PDF

Then PDF file should be downloaded

And file extension should be ".pdf"

And report data should be accurate

Pagination

Scenario: Verify pagination

Given report contains multiple pages

When user clicks Next Page

Then next set of records should be displayed

When user clicks Previous Page
Then previous records should be displayed

Filters

Scenario: Verify report filters
Given user is on Reports page
When user applies Status filter
And clicks Search
Then only matching records should appear

Critical Test Scenarios

1. Report generation with invalid date range.
2. Exported file content validation.
3. Data consistency between UI and database/API.
4. Large volume report generation.
5. Role-based report visibility.
6. Performance under concurrent users.
7. Session timeout handling.
8. Error handling during backend failure.
9. Sorting, filtering and pagination accuracy.
10. Download functionality for PDF, Excel and CSV.

If this is for a **Supply Chain or eCommerce application**, I can generate **50-100 domain-specific report test scripts** (Orders, Inventory, Shipment, Returns, Sales, Vendor, Invoice reports) in a detailed format including **Priority, Preconditions, Test Data, Expected Results, and Automation Feasibility**.

Prompt using ICEPOT:

Context:

Act as a Senior Automation Test Engineer for a Supply chain domain and e-commerce application

Instruction:

[MANDATORY] Generate Selenium Java Script with Selenium version 4.0 in the page object model

[MANDATORY] Use TestNG annotations

[STRICT] Generate only the code no other text

[MANDATORY] Generate test scripts to validate Excel reports

[MANDATORY] Should cover date range filters, Pass/Fail filters, Execution time

[MANDATORY] Add Explicit waits

[CRITICAL] Do not add thread.sleep

[HIGHPRIORITY] Implement Retry mechanism

[STRICT] Should not print duplicate test case information when triggered Retry for a test case

[HIGHPRIORITY] Implement Exception Handling

[MANDATORY] Add Assertions

[LOWPRIORITY] Add comments to explain the code

Example:

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.testng.Assert;
import org.testng.annotations.*;

import java.time.Duration;

public class SalesforceLoginTest {

    WebDriver driver;
```

```
WebDriverWait wait;
```

```
@BeforeMethod
```

```
public void setup() {  
    driver = new EdgeDriver();  
    driver.manage().window().maximize();  
    wait = new WebDriverWait(driver, Duration.ofSeconds(15));  
}
```

```
@Test
```

```
public void verifySalesforceLogin() {
```

```
    driver.get("https://login.salesforce.com/");
```

```
    WebElement username = wait.until(  
        ExpectedConditions.visibilityOfElementLocated(By.id("username"))  
    );
```

```
    username.sendKeys("your_username");
```

```
    WebElement password = driver.findElement(By.id("password"));
```

```
    password.sendKeys("your_password");
```

```
    WebElement loginButton = driver.findElement(By.id("Login"));
```

```
    loginButton.click();
```

```
    wait.until(ExpectedConditions.or(  
        ExpectedConditions.urlContains("lightning"),  
        ExpectedConditions.urlContains("setup")  
    ));
```

```
));
```

```
Assert.assertTrue(  
    driver.getCurrentUrl().contains("salesforce.com"),  
    "Login failed or Salesforce home page not loaded"  
);  
}
```

```
@AfterMethod
```

```
public void tearDown() {  
    if (driver != null) {  
        driver.quit();  
    }  
}  
}
```

Persona:

Act as a Automation lead to review the testcases for accuracy and provide a score

Output:

Code should be a downloadable text file

Tone:

Generate the output in a professional and concise manner, focusing on the technical aspects of the code.

Output:



Selenium_Excel_Repo
rt_Test_Scripts.txt